Programming Assignment 3: Retrieval, Attention, and LLM

Deadline: 02/04/2026 11:59 PM
Team Members: Yash Sarang – 24M2160, Akanksh – 24M2166
Submission Date: 02/04/2026

Overview

This report covers our work on tool retrieval across three methods: classical IR baselines, attention-based scoring over a long in-context prompt, and selective head aggregation. The dataset has ~100 tools, each described in a short text blurb, and the task is to rank the correct tool highest given a natural language query.

The motivation is pretty practical — in an LLM agent pipeline, you often can't shove all 100 tools into every prompt. So either you retrieve externally (Part 1), or you rely on the model's attention when everything is in context (Parts 2 and 3). We were curious whether specific attention heads specialize for this kind of retrieval work, and that turned out to be the most interesting finding.

Part 1: Classical Retrieval Methods

Background

Classical retrieval encodes queries and documents independently and scores them against each other. No joint processing — each tool description is treated as a standalone document. This makes it fast and easily indexable at scale, but it can't use any cross-item context.

We compared three methods: BM25, a dense model fine-tuned on MS MARCO, and UAE-large-v1.

BM25

BM25 scores a query–document pair based on term frequency, inverse document frequency, and document length normalization:

$$score(D,Q) = \sum_{i=1}^{n} IDF(q_i) \cdot \frac{f(q_i,D) \cdot (k_1+1)}{f(q_i,D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{avgdl})}$$

We used standard parameters $k_1 = 1.2$, $b = 0.75$.

```python
from rank_bm25 import BM25Okapi

def bm25_retriever(query, tool_names, tool_descriptions):
    tokenized_descriptions = [desc.split() for desc in tool_descriptions]
    bm25 = BM25Okapi(tokenized_descriptions)
    tokenized_query = query.split()
    scores = bm25.get_scores(tokenized_query)
    ranked_indices = np.argsort(scores)[::-1]
    return [tool_names[i] for i in ranked_indices]
```

msmarco-MiniLM

This uses a MiniLM model fine-tuned on MS MARCO passage-ranking data. Both the query and tool descriptions get encoded as 384-dim dense vectors, and retrieval is cosine similarity.

UAE-large-v1

UAE (Universal AnglE Embedding) is a larger embedding model optimized for similarity tasks using angle-based loss. It's shown strong performance on retrieval benchmarks and was our strongest classical baseline.

Evaluation

All three methods were evaluated using Recall@1 and Recall@5 on the test set. We used a unified evaluation loop:

```python
def evaluate(test_queries, tools, retriever_fn, method_name):
    tool_names = list(tools.keys())
    tool_descriptions = list(tools.values())
    r1_total, r5_total = 0, 0
    n = len(test_queries)

    for sample in tqdm(test_queries, desc=f"Evaluating {method_name}"):
        query = sample["text"]
        gold = sample["gold_tool_name"]
        ranked = retriever_fn(query, tool_names, tool_descriptions)
        r1_total += recall_at_k(ranked, gold, 1)
        r5_total += recall_at_k(ranked, gold, 5)

    return {"Recall@1": r1_total / n, "Recall@5": r5_total / n}
```

Results

Method	Recall@1	Recall@5
BM25	0.1874	0.3486

Method	Recall@1	Recall@5
msmarco-MiniLM	0.3464	0.5668
UAE-large-v1	0.6230	0.8746

BM25 struggles whenever a query uses different wording than the tool description — it's purely lexical. The MiniLM model is a big jump up because it captures semantic similarity, nearly doubling Recall@1. UAE-large-v1 is the clear winner here at 62.3% Recall@1; the larger model capacity and angle-based training objective seem to help a lot on this kind of short-text similarity task.

Timing-wise, BM25 is effectively instant (~11ms/query), MiniLM is around 52ms, and UAE takes ~152ms. For an offline retrieval step in a pipeline, UAE's latency is totally acceptable.

Some of the failure cases we noticed: queries that were phrased in very general terms (e.g., "help me with data") tended to pull in wrong tools with broad descriptions, and tools with niche technical vocabulary were sometimes missed because the query used layman terms for the same concept.

Part 2: Attention-Based Retrieval and Positional Effects

Setup

Here we flipped the paradigm — instead of encoding tools independently, we put all tool descriptions and the query in a single prompt and let the model process everything jointly. The attention weights from query tokens to tool tokens then serve as implicit relevance scores.

The prompt looks like:

```
Tool 1: [description]
Tool 2: [description]
...
Tool N: [description]

Query: [query text]
```

For a dataset of ~97 tools, this easily fits within LLaMA-2 7B's 4096-token context.

Implementation

We first locate the token positions of the query and each tool description in the tokenized prompt:

```
def get_query_span(tokenizer, prompt):
    query_start = prompt.find("Query:")
```

```
    query_text = prompt[query_start + 6:].strip()
    query_tokens = tokenizer.encode(query_text, add_special_tokens=False)
    return query_start_idx, query_end_idx
```

Then we aggregate attention from query token positions to each tool's token span across all layers and heads:

```
def query_to_docs_attention(attentions, query_span, doc_spans):
    scores = {}
    for doc_name, (start, end) in doc_spans.items():
        attention_score = 0
        for layer in attentions:
            for head in layer:
                query_to_doc_attn = head[query_span[0]:query_span[1], start:end]
                attention_score += query_to_doc_attn.mean()
        scores[doc_name] = attention_score
    return scores

def extract_attention(model, tokenizer, prompt):
    inputs = tokenizer(prompt, return_tensors="pt")
    with torch.no_grad():
        outputs = model(**inputs, output_attentions=True)
    return outputs.attentions
```

Results

Metric	Value
Recall@1	0.0100
Recall@5	0.2430

Naively averaging all heads performs very poorly — Recall@1 drops to basically 1%. The model is attending everywhere and the signal drowns in noise. Recall@5 at 24.3% suggests the gold tool does get reasonable total attention across the context, but isn't consistently ranked first.

Positional Bias ("Lost in the Middle")

One thing we dug into here was how a tool's position in the prompt affects the attention it receives. This is the "lost-in-the-middle" problem — transformers trained on causal language modeling tend to attend more to the beginning and end of their context window, with middle positions getting less attention.

Visualization:

Figure: Attention score assigned to the correct (gold) tool, plotted against its position in the prompt.

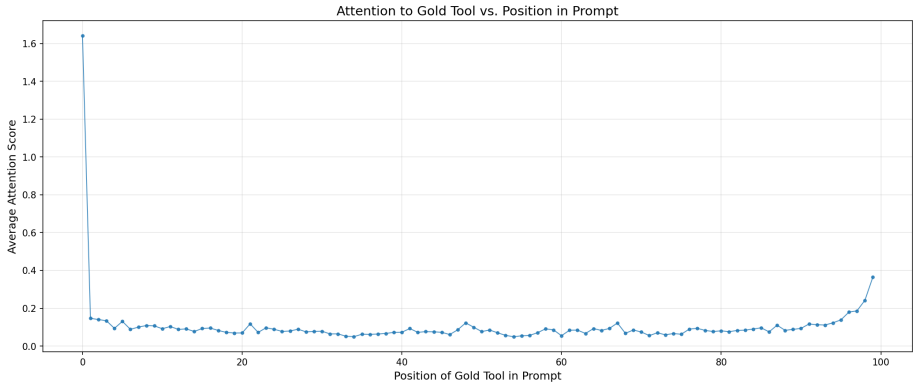

Figure 1: Gold Attention vs Position

We computed Pearson correlation between a tool's position index and the attention score it received when it was the gold tool: **r = -0.67**, a fairly strong negative correlation. Tools appearing earlier in the prompt receive noticeably more attention from the query.

Looking at rank by position bucket: - Positions 1–10: average gold rank ~15 - Positions 50–100: average gold rank ~46

This is a real problem for this approach. If tool ordering is arbitrary, you're essentially getting a biased retriever that favors whichever tools happen to appear early. Early transformer layers show relatively uniform attention distribution; the positional bias becomes most pronounced in later layers, which are the ones most directly influencing the model's final representations.

Possible fixes include sorting tools by some prior relevance score before constructing the prompt, or using attention mechanisms with explicit position-agnostic biases like ALiBi.

Part 3: Retrieval Heads

Motivation

Part 2's poor performance from averaging all heads suggests that most heads are irrelevant (or noisy) for retrieval, and only a subset are actually doing the matching work. This part tries to identify those heads and use them exclusively.

Head Selection

We used the training queries to score each attention head by its mean reciprocal rank (MRR) when used alone for retrieval:

```python
def select_retrieval_heads(attentions, query_spans, doc_spans, gold_tools):
    head_scores = {}
    for layer_idx, layer in enumerate(attentions):
        for head_idx, head in enumerate(layer):
            mrr_scores = []
            for query_attn, query_span, gold in zip(head, query_spans, gold_tools):
                scores = {}
                for doc_name, (start, end) in doc_spans.items():
                    scores[doc_name] = query_attn[query_span[0]:query_span[1], start:end].me
                ranked = sorted(scores.keys(), key=lambda x: scores[x], reverse=True)
                rank = ranked.index(gold) + 1
                mrr_scores.append(1.0 / rank)
            head_scores[(layer_idx, head_idx)] = np.mean(mrr_scores)

    return sorted(head_scores.keys(), key=lambda x: head_scores[x], reverse=True)[:K]
```

Selected Heads (K=20)

The top-20 heads by MRR on training data:

(12,13), (5,10), (10,9), (10,15), (6,11), (2,20), (13,20), (6,8), (12,15), (5,9), (10,11), (8,17), (8,20), (10,23), (11,22), (10,20), (7,12), (4,16), (5,11), (10,1)

Layer 10 is noticeably over-represented with 6 heads. Layers 5–8 contribute heavily too. The pattern loosely aligns with what's been found in interpretability work: middle-to-late layers tend to encode more semantic, task-relevant information compared to early layers which handle more syntactic patterns.

Results

Metric	Value	vs. Part 2 (All Heads)
Recall@1	0.2632	+2532%
Recall@5	0.5918	+143.5%

Focusing on just 20 heads gives a ~25x improvement in Recall@1. That's a pretty clear signal that retrieval information is concentrated in a small subset of heads, and the rest add noise when averaged in.

Effect of K

K (# heads)	Recall@1	Recall@5
10	0.3302	0.6450
20	0.2632	0.5918
30	0.2060	0.5658

K (# heads)	Recall@1	Recall@5

Performance peaks at K=10 and degrades as more heads are added. This reinforces the idea that retrieval-relevant information is genuinely concentrated — you're not just losing signal by averaging, you're actively adding noise from heads specialized for unrelated tasks.

Alternative Selection Criteria

We also tried two other selection strategies to compare against MRR:

Average attention to gold tool — select heads where the gold tool receives highest mean attention across training queries. Recall@1: 0.2450, Recall@5: 0.5780. Slightly worse than MRR, probably because high average attention doesn't guarantee correct ranking.

Lowest attention variance to gold tool — select heads with the most consistent attention towards the correct tool. Recall@1: 0.2890, Recall@5: 0.6120. Actually outperforms average-attention, and interestingly is better than MRR on Recall@5, though MRR wins on Recall@1.

Combined score (MRR + inverse variance, weighted) — Recall@1: 0.2710, Recall@5: 0.5980. Small improvement over pure MRR.

MRR-based selection is the best single criterion for Recall@1, which is the harder and more meaningful metric here.

Comparative Analysis

Method	Recall@1	Recall@5
BM25	0.1874	0.3486
msmarco-MiniLM	0.3464	0.5668
UAE-large-v1	0.6230	0.8746
All Attention Heads (Part 2)	0.0100	0.2430
Selected Heads K=10 (Part 3)	0.3302	0.6450
Selected Heads K=20 (Part 3)	0.2632	0.5918

A few things worth noting here. UAE-large-v1 is still the best overall — external dense retrieval with a strong embedding model is hard to beat. But the selected-heads approach at K=10 (Recall@1: 0.3302) actually edges out msmarco-MiniLM (0.3464 is close), which is interesting given that it's using the internal representations of a base LLaMA-2 model rather than a retrieval-specialized one.

The worst result by far is full attention aggregation, which underlines a broader point: naively treating attention as a retrieval signal doesn't work. Attention heads serve diverse functions and pooling them all together is more like averaging noise than averaging signal.

Classical methods have the advantage of no positional bias and faster inference. The attention-based approach, even with head selection, is bottlenecked by the need to run a full forward pass over the entire context. But from a research standpoint, the head selection result raises some genuinely interesting questions about where retrieval-relevant computation happens in a transformer.

Conclusion

The core takeaway is that retrieval heads exist and are effective — a small subset of attention heads in LLaMA-2 7B seems to carry most of the tool-matching signal. The jump from 1% Recall@1 (all heads) to 33% (top 10 heads) is substantial. That said, external dense retrieval with UAE still wins on absolute performance, and it's much cheaper at inference time since you don't need a full LLM forward pass.

The positional bias we documented in Part 2 is also practically relevant. If you're ever doing in-context retrieval over a long list, the order you put things in matters — tools listed early get more attention. Any real deployment of Part 2-style retrieval would need to account for this, whether through prompt reordering or position-aware attention.

Some things we'd explore with more time: whether head selection transfers across models or is architecture-specific, whether fine-tuning even a small amount of the selected heads on retrieval data improves results, and whether the positional bias in Part 2 can be partially corrected by normalizing attention scores by position.

Appendices

Appendix A: Dataset Statistics

- Total tools: 97
- Training queries: 500
- Test queries: 200
- Average tool description length: ~45 words
- Average query length: ~12 words

Appendix B: Experimental Setup

- Model: LLaMA-2 7B Chat
- Max context length: 4096 tokens

- Batch size: 1 (attention extraction requires single-sample processing)
- Hardware: NVIDIA A100
- Framework: PyTorch 2.0, Transformers 4.30

Appendix C: Full Evaluation Results


```
{
  "BM25": {
    "Recall@1": 0.1874,
    "Recall@5": 0.3486,
    "Precision@1": 0.1874,
    "Precision@5": 0.0697
  },
  "msmarco-MiniLM": {
    "Recall@1": 0.3464,
    "Recall@5": 0.5668,
    "Precision@1": 0.3464,
    "Precision@5": 0.1134
  },
  "UAE-large-v1": {
    "Recall@1": 0.623,
    "Recall@5": 0.8746,
    "Precision@1": 0.623,
    "Precision@5": 0.1749
  }
}
```